

# Multi-Tenant SaaS Product Database Architecture

## Table of Contents

1. [Overview](#)
  2. [Requirements](#)
  3. [Capacity Estimation](#)
  4. [Schema Design](#)
  5. [ASCII ER Diagram](#)
  6. [Indexing Strategy](#)
  7. [Query Patterns](#)
  8. [Scaling Strategy](#)
  9. [Tradeoffs](#)
  10. [Interview Discussion Points](#)
  11. [Common Interview Follow-ups](#)
  12. [Performance Considerations](#)
  13. [Cross-References](#)
- 

## Overview

Multi-tenant SaaS introduces a fundamental design decision that affects every other decision: how do you isolate tenant data? This document covers the shared-schema approach (one row per tenant, `org_id` everywhere) which is the most common choice for B2B SaaS at scale. It includes organization and user management, subscription and billing, feature flags, and usage metering — the complete foundation of a modern SaaS product.

---

## Requirements

### Functional Requirements

- Organizations (tenants) subscribe to plans
- Multiple users per organization with role-based access control
- Subscription management: free trial → paid → upgrade → downgrade → cancellation
- Feature flags: per-plan features, per-org overrides, gradual rollouts
- Usage metering: count API calls, storage, seats — for billing and enforcement
- Billing: monthly/annual, seat-based or usage-based, invoices, receipts
- Audit trail: who did what in the product
- SSO integration (SAML/OIDC) for enterprise customers
- Organization-level customization and settings

### Non-Functional Requirements

- 100K+ organizations
  - 5M+ users (avg 50 per org)
  - Row-level security: users can never see another org's data
  - 99.9% uptime
  - Billing system idempotent and retry-safe
  - Feature flag evaluations: sub-millisecond
  - Usage metering: eventually consistent (slight lag acceptable)
-

## Capacity Estimation

| Metric                           | Estimate |
|----------------------------------|----------|
| Organizations                    | 100K     |
| Users                            | 5M       |
| API calls metered per day        | 1B       |
| Usage events per day             | 500M     |
| Feature flag evaluations per day | 10B      |
| Invoices generated per month     | 100K     |
| Audit log events per day         | 100M     |

## Storage Estimates

| Table           | Rows           | Row Size | Total     |
|-----------------|----------------|----------|-----------|
| organizations   | 100K           | 1KB      | 100MB     |
| users           | 5M             | 600B     | 3GB       |
| subscriptions   | 300K (history) | 500B     | 150MB     |
| usage_events    | 15B/yr         | 200B     | 3TB/yr    |
| usage_summaries | 1.2M/mo        | 300B     | 4GB/yr    |
| audit_events    | 36.5B/yr       | 400B     | 14.6TB/yr |
| feature_flags   | 1K             | 2KB      | 2MB       |

**Critical insight:** usage\_events and audit\_events are write-heavy and time-series in nature. These are good candidates for TimescaleDB or columnar storage. The schema below keeps them in PostgreSQL with aggressive partitioning.

## Schema Design

```
-- =====
-- EXTENSIONS
-- =====

CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";

-- =====
-- ENUM TYPES
-- =====

CREATE TYPE org_status AS ENUM ('trial', 'active', 'past_due', 'suspended',
```

```

'cancelled');
CREATE TYPE user_status AS ENUM ('invited', 'active', 'suspended', 'removed');
CREATE TYPE member_role AS ENUM ('owner', 'admin', 'member', 'viewer',
'billing_admin');
CREATE TYPE plan_tier AS ENUM ('free', 'starter', 'growth', 'business',
'enterprise');
CREATE TYPE billing_interval AS ENUM ('monthly', 'annual');
CREATE TYPE invoice_status AS ENUM ('draft', 'open', 'paid', 'void',
'uncollectible');
CREATE TYPE flag_type AS ENUM ('boolean', 'string', 'number', 'json');
CREATE TYPE rollout_type AS ENUM ('all', 'percentage', 'org_list', 'plan_list');

-- =====
-- ORGANIZATIONS (TENANTS)
-- =====
CREATE TABLE organizations (
    org_id          UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    name            VARCHAR(300)  NOT NULL,
    slug            VARCHAR(200)  NOT NULL,
    domain          VARCHAR(300),  -- verified domain for SSO
    logo_url        VARCHAR(500),
    status          org_status    NOT NULL DEFAULT 'trial',
    plan            plan_tier     NOT NULL DEFAULT 'free',
    trial_ends_at   TIMESTAMPTZ,
    country_code    CHAR(2),
    timezone        VARCHAR(60)   NOT NULL DEFAULT 'UTC',
    settings        JSONB         NOT NULL DEFAULT '{}',
    metadata        JSONB         NOT NULL DEFAULT '{}',
    created_at      TIMESTAMPTZ   NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ   NOT NULL DEFAULT now(),
    CONSTRAINT uq_org_slug UNIQUE (slug),
    CONSTRAINT uq_org_domain UNIQUE (domain)
);

-- =====
-- USERS AND MEMBERSHIP
-- =====
CREATE TABLE users (
    user_id         UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    email           VARCHAR(320)  NOT NULL,
    full_name       VARCHAR(200)  NOT NULL,
    avatar_url      VARCHAR(500),
    password_hash   VARCHAR(255),
    status          user_status   NOT NULL DEFAULT 'invited',
    email_verified  BOOLEAN       NOT NULL DEFAULT FALSE,
    mfa_enabled     BOOLEAN       NOT NULL DEFAULT FALSE,
    mfa_secret      VARCHAR(200),  -- TOTP secret (encrypted at rest)
    last_login_at   TIMESTAMPTZ,
    created_at      TIMESTAMPTZ   NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ   NOT NULL DEFAULT now(),
    CONSTRAINT uq_users_email UNIQUE (email)
);

```

```

-- A user can belong to multiple organizations
CREATE TABLE org_members (
    member_id        UUID                PRIMARY KEY DEFAULT uuid_generate_v4(),
    org_id            UUID                NOT NULL REFERENCES organizations(org_id) ON
DELETE CASCADE,
    user_id           UUID                NOT NULL REFERENCES users(user_id) ON DELETE
CASCADE,
    role              member_role        NOT NULL DEFAULT 'member',
    invited_by        UUID                REFERENCES users(user_id),
    invited_at        TIMESTAMPTZ        NOT NULL DEFAULT now(),
    joined_at         TIMESTAMPTZ,
    removed_at        TIMESTAMPTZ,
    is_active         BOOLEAN            NOT NULL DEFAULT TRUE,
    CONSTRAINT uq_org_member UNIQUE (org_id, user_id)
);

-- SSO configuration per organization
CREATE TABLE sso_configs (
    sso_id            UUID                PRIMARY KEY DEFAULT uuid_generate_v4(),
    org_id            UUID                NOT NULL REFERENCES organizations(org_id) ON
DELETE CASCADE,
    provider          VARCHAR(50)        NOT NULL,    -- 'saml', 'oidc', 'google',
'microsoft'
    metadata_url      VARCHAR(1000),
    client_id         VARCHAR(500),
    client_secret     VARCHAR(500),    -- encrypted
    attribute_mapping JSONB,           -- maps IdP attributes to user fields
    is_active         BOOLEAN            NOT NULL DEFAULT TRUE,
    created_at        TIMESTAMPTZ        NOT NULL DEFAULT now(),
    CONSTRAINT uq_sso_org UNIQUE (org_id, provider)
);

-- =====
-- PLANS AND SUBSCRIPTIONS
-- =====

CREATE TABLE plans (
    plan_id           SERIAL              PRIMARY KEY,
    tier               plan_tier           NOT NULL,
    name              VARCHAR(100)        NOT NULL,
    billing_interval  billing_interval    NOT NULL,
    price_cents       INT                 NOT NULL,
    currency          CHAR(3)             NOT NULL DEFAULT 'USD',
    seat_limit        INT,                 -- NULL = unlimited
    api_call_limit    BIGINT,              -- NULL = unlimited
    storage_gb_limit  INT,                 -- NULL = unlimited
    is_active         BOOLEAN            NOT NULL DEFAULT TRUE,
    created_at        TIMESTAMPTZ        NOT NULL DEFAULT now(),
    CONSTRAINT uq_plan_tier_interval UNIQUE (tier, billing_interval)
);

CREATE TABLE subscriptions (

```

```

sub_id          UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
org_id          UUID          NOT NULL REFERENCES organizations(org_id),
plan_id         INT           NOT NULL REFERENCES plans(plan_id),
status          org_status    NOT NULL DEFAULT 'active',
billing_interval billing_interval NOT NULL,
current_period_start DATE      NOT NULL,
current_period_end DATE      NOT NULL,
trial_start     DATE,
trial_end       DATE,
quantity        INT           NOT NULL DEFAULT 1, -- seats
stripe_sub_id   VARCHAR(200), -- external billing provider reference
cancelled_at     TIMESTAMPTZ,
cancel_at_period_end BOOLEAN   NOT NULL DEFAULT FALSE,
created_at      TIMESTAMPTZ    NOT NULL DEFAULT now(),
updated_at      TIMESTAMPTZ    NOT NULL DEFAULT now(),
CONSTRAINT uq_org_active_sub UNIQUE (org_id) -- one active sub per org
);

CREATE TABLE subscription_history (
  history_id      UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
  org_id          UUID          NOT NULL REFERENCES organizations(org_id),
  sub_id          UUID          NOT NULL REFERENCES subscriptions(sub_id),
  event_type      VARCHAR(50)    NOT NULL, --
  'created', 'upgraded', 'downgraded', 'cancelled', 'renewed'
  from_plan_id    INT           REFERENCES plans(plan_id),
  to_plan_id      INT           REFERENCES plans(plan_id),
  changed_by      UUID          REFERENCES users(user_id),
  created_at      TIMESTAMPTZ    NOT NULL DEFAULT now()
);

-- =====
-- BILLING
-- =====

CREATE TABLE payment_methods (
  pm_id          UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
  org_id          UUID          NOT NULL REFERENCES organizations(org_id),
  provider        VARCHAR(50)    NOT NULL DEFAULT 'stripe',
  provider_pm_id  VARCHAR(200)    NOT NULL, -- Stripe payment method ID
  last_four       CHAR(4),
  expiry_month    SMALLINT,
  expiry_year     SMALLINT,
  card_brand      VARCHAR(30),
  billing_name    VARCHAR(200),
  billing_email   VARCHAR(320),
  is_default      BOOLEAN        NOT NULL DEFAULT FALSE,
  created_at      TIMESTAMPTZ    NOT NULL DEFAULT now()
);

CREATE TABLE invoices (
  invoice_id      UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
  org_id          UUID          NOT NULL REFERENCES organizations(org_id),
  sub_id          UUID          REFERENCES subscriptions(sub_id),

```

```

invoice_number VARCHAR(50) NOT NULL,
status         invoice_status NOT NULL DEFAULT 'draft',
period_start   DATE           NOT NULL,
period_end     DATE           NOT NULL,
subtotal_cents INT            NOT NULL DEFAULT 0,
discount_cents INT            NOT NULL DEFAULT 0,
tax_cents      INT            NOT NULL DEFAULT 0,
total_cents    INT            NOT NULL DEFAULT 0,
currency       CHAR(3)        NOT NULL DEFAULT 'USD',
pm_id          UUID           REFERENCES payment_methods(pm_id),
provider_invoice_id VARCHAR(200),
paid_at        TIMESTAMPTZ,
due_date       DATE,
created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
CONSTRAINT uq_invoice_number UNIQUE (invoice_number)
) PARTITION BY RANGE (period_start);

CREATE TABLE invoice_line_items (
  line_id          UUID           PRIMARY KEY DEFAULT uuid_generate_v4(),
  invoice_id       UUID           NOT NULL REFERENCES invoices(invoice_id),
  description      VARCHAR(500)   NOT NULL,
  quantity         NUMERIC(12,4)  NOT NULL DEFAULT 1,
  unit_price_cents INT            NOT NULL,
  amount_cents     INT            NOT NULL,
  metadata         JSONB
);

-- =====
-- FEATURE FLAGS
-- =====

CREATE TABLE feature_flags (
  flag_id          UUID           PRIMARY KEY DEFAULT uuid_generate_v4(),
  key              VARCHAR(200)   NOT NULL,
  name             VARCHAR(300)   NOT NULL,
  description      TEXT,
  flag_type        flag_type      NOT NULL DEFAULT 'boolean',
  default_value    JSONB          NOT NULL DEFAULT 'false',
  rollout_type     rollout_type   NOT NULL DEFAULT 'all',
  rollout_pct      NUMERIC(5,2),  -- 0-100 for percentage rollout
  rollout_plan_list plan_tier[],  -- plans that get this flag
  rollout_org_list UUID[],        -- specific orgs (use sparingly)
  is_active        BOOLEAN        NOT NULL DEFAULT TRUE,
  deprecated_at    TIMESTAMPTZ,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_flag_key UNIQUE (key)
);

-- Per-org flag overrides (for enterprise custom flags)
CREATE TABLE flag_overrides (
  override_id      UUID           PRIMARY KEY DEFAULT uuid_generate_v4(),
  flag_id          UUID           NOT NULL REFERENCES feature_flags(flag_id) ON

```

```

DELETE CASCADE,
    org_id          UUID          NOT NULL REFERENCES organizations(org_id) ON
DELETE CASCADE,
    value           JSONB         NOT NULL,
    reason          TEXT,
    created_by      UUID          REFERENCES users(user_id),
    created_at      TIMESTAMPTZ    NOT NULL DEFAULT now(),
    expires_at      TIMESTAMPTZ,
    CONSTRAINT uq_flag_org_override UNIQUE (flag_id, org_id)
);

-- =====
-- USAGE METERING
-- =====
-- Raw events (high volume – kept for 90 days, then summarized)
CREATE TABLE usage_events (
    event_id        UUID          NOT NULL DEFAULT uuid_generate_v4(),
    org_id          UUID          NOT NULL REFERENCES organizations(org_id),
    user_id         UUID          REFERENCES users(user_id),
    metric          VARCHAR(100)  NOT NULL, --
'api_calls','storage_bytes','seats_used'
    quantity        NUMERIC(20,4) NOT NULL    DEFAULT 1,
    properties      JSONB,
    occurred_at     TIMESTAMPTZ    NOT NULL DEFAULT now(),
    PRIMARY KEY (event_id, occurred_at)
) PARTITION BY RANGE (occurred_at);

-- Hourly summaries (long-term retention)
CREATE TABLE usage_summaries (
    org_id          UUID          NOT NULL REFERENCES organizations(org_id),
    metric          VARCHAR(100)  NOT NULL,
    period_start    TIMESTAMPTZ    NOT NULL, -- truncated to hour
    period_end      TIMESTAMPTZ    NOT NULL,
    total_quantity  NUMERIC(20,4) NOT NULL,
    event_count     INT           NOT NULL,
    created_at      TIMESTAMPTZ    NOT NULL DEFAULT now(),
    PRIMARY KEY (org_id, metric, period_start)
) PARTITION BY RANGE (period_start);

-- Current billing period usage (for quota enforcement – hot read)
CREATE TABLE usage_current_period (
    org_id          UUID          NOT NULL REFERENCES organizations(org_id),
    metric          VARCHAR(100)  NOT NULL,
    period_start    DATE          NOT NULL,
    quantity        NUMERIC(20,4) NOT NULL DEFAULT 0,
    updated_at      TIMESTAMPTZ    NOT NULL DEFAULT now(),
    PRIMARY KEY (org_id, metric, period_start)
);

-- =====
-- AUDIT LOG
-- =====

```

```

CREATE TABLE audit_events (
  event_id      UUID          NOT NULL DEFAULT uuid_generate_v4(),
  org_id        UUID          NOT NULL,
  actor_id      UUID,         -- user_id or NULL for system
  actor_type     VARCHAR(20)   NOT NULL DEFAULT 'user',
  action        VARCHAR(200)   NOT NULL, --
  'user.created', 'subscription.upgraded'
  resource_type  VARCHAR(100),
  resource_id    UUID,
  ip_address     INET,
  user_agent     TEXT,
  metadata      JSONB,
  occurred_at    TIMESTAMPTZ   NOT NULL DEFAULT now(),
  PRIMARY KEY (event_id, occurred_at)
) PARTITION BY RANGE (occurred_at);

-- Automate partition creation (shown conceptually)
CREATE TABLE audit_events_2024_01 PARTITION OF audit_events
  FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

-- =====
-- ROW-LEVEL SECURITY
-- =====
-- Enable RLS on all multi-tenant tables
ALTER TABLE org_members ENABLE ROW LEVEL SECURITY;
ALTER TABLE usage_events ENABLE ROW LEVEL SECURITY;
ALTER TABLE audit_events ENABLE ROW LEVEL SECURITY;
ALTER TABLE invoices ENABLE ROW LEVEL SECURITY;

-- Policy: users can only see their own org's data
-- Application sets: SET LOCAL app.current_org_id = $org_id
CREATE POLICY org_isolation_policy ON org_members
  USING (org_id = current_setting('app.current_org_id')::UUID);

CREATE POLICY org_isolation_policy ON usage_events
  USING (org_id = current_setting('app.current_org_id')::UUID);

-- Admin role bypasses RLS
CREATE POLICY admin_bypass ON org_members
  TO app_admin USING (TRUE);

-- =====
-- FEATURE FLAG EVALUATION FUNCTION
-- =====
CREATE OR REPLACE FUNCTION evaluate_flag(
  p_flag_key  VARCHAR,
  p_org_id    UUID,
  p_plan      plan_tier
) RETURNS JSONB AS $$
DECLARE
  v_flag      feature_flags%ROWTYPE;
  v_override  flag_overrides%ROWTYPE;

```



```

BEGIN
-- Get flag definition
SELECT * INTO v_flag FROM feature_flags WHERE key = p_flag_key AND is_active;
IF NOT FOUND THEN RETURN 'false'::JSONB; END IF;

-- Check for org-level override first
SELECT * INTO v_override
FROM flag_overrides
WHERE flag_id = v_flag.flag_id
AND org_id = p_org_id
AND (expires_at IS NULL OR expires_at > now());
IF FOUND THEN RETURN v_override.value; END IF;

-- Evaluate rollout rules
CASE v_flag.rollout_type
WHEN 'all' THEN
RETURN v_flag.default_value;
WHEN 'plan_list' THEN
IF p_plan = ANY(v_flag.rollout_plan_list) THEN
RETURN 'true'::JSONB;
ELSE
RETURN 'false'::JSONB;
END IF;
WHEN 'org_list' THEN
IF p_org_id = ANY(v_flag.rollout_org_list) THEN
RETURN 'true'::JSONB;
ELSE
RETURN 'false'::JSONB;
END IF;
WHEN 'percentage' THEN
-- Deterministic hash-based rollout
IF (('x' || substr(md5(p_org_id::text || p_flag_key), 1,
8))::bit(32)::int::bigint + 2147483648) % 100
< v_flag.rollout_pct THEN
RETURN 'true'::JSONB;
ELSE
RETURN 'false'::JSONB;
END IF;
ELSE
RETURN v_flag.default_value;
END CASE;
END;
$$ LANGUAGE plpgsql STABLE;

```

## ASCII ER Diagram

```

+-----+      +-----+      +-----+
| organizations |1-----N|org_members|N-----1|   users   |
+-----+      +-----+      +-----+
|1

```

```

      |N
+-----+      +-----+      +-----+
|subscriptions|1---1| invoices |1-----N| invoice_line_items|
+-----+      +-----+      +-----+

      |1
      |N
+-----+      +-----+
|sub_history|      | plans      |
+-----+      +-----+

+-----+      +-----+      +-----+
| organizations |1-----N| flag_ovrrd |N-----1| feature_flags |
+-----+      +-----+      +-----+

+-----+      +-----+
| organizations |1-----N| usage_events |
+-----+      +-----+

      |1
      |N      +-----+
      +-----+ | usage_summaries |
      |      +-----+
      |N
+-----+
| usage_current_per |
+-----+

+-----+      +-----+
| organizations |1-----N| audit_events |
+-----+      +-----+

+-----+      +-----+
| organizations |1-----1| sso_configs |
+-----+      +-----+

```

## Indexing Strategy

```

-- Organizations: slug and domain lookup (auth flow)
CREATE INDEX idx_orgs_slug      ON organizations (slug);
CREATE INDEX idx_orgs_domain    ON organizations (domain) WHERE domain IS NOT NULL;
CREATE INDEX idx_orgs_status    ON organizations (status) WHERE status NOT IN
('cancelled');

-- Users: email lookup (authentication)
CREATE INDEX idx_users_email     ON users (lower(email));

-- Org members: user's orgs and org's members
CREATE INDEX idx_members_user    ON org_members (user_id) WHERE is_active = TRUE;
CREATE INDEX idx_members_org     ON org_members (org_id, role) WHERE is_active =
TRUE;

```

```

-- Subscriptions: billing jobs
CREATE INDEX idx_subs_renewal ON subscriptions (current_period_end, status)
WHERE status = 'active';

-- Invoices: org billing history
CREATE INDEX idx_invoices_org ON invoices (org_id, period_start DESC);
CREATE INDEX idx_invoices_open ON invoices (status, due_date)
WHERE status IN ('open', 'past_due');

-- Feature flags: evaluation hot path (likely fully cached in Redis)
CREATE INDEX idx_flags_key ON feature_flags (key) WHERE is_active = TRUE;
CREATE INDEX idx_overrides_org ON flag_overrides (org_id, flag_id)
WHERE expires_at IS NULL OR expires_at > now();

-- Usage events: org + time range (for summary jobs)
CREATE INDEX idx_usage_events_org_time ON usage_events (org_id, occurred_at DESC);
CREATE INDEX idx_usage_events_metric ON usage_events (metric, occurred_at DESC);

-- Usage current period: quota enforcement (very hot read)
CREATE INDEX idx_usage_current_org ON usage_current_period (org_id, metric);

-- Audit events: org-filtered event log
CREATE INDEX idx_audit_org_time ON audit_events (org_id, occurred_at DESC);
CREATE INDEX idx_audit_actor ON audit_events (actor_id, occurred_at DESC)
WHERE actor_id IS NOT NULL;

```

## Query Patterns

### Q1 — Feature Flag Evaluation (via function, cached in Redis)

```

-- Bulk flag evaluation for a session
SELECT
  ff.key,
  evaluate_flag(ff.key, $org_id::UUID, $plan::plan_tier) AS value
FROM feature_flags ff
WHERE ff.is_active = TRUE;

```

### Q2 — User Authentication and Org Loading

```

SELECT
  u.user_id, u.email, u.full_name, u.password_hash,
  u.mfa_enabled, u.status,
  json_agg(jsonb_build_object(
    'org_id', om.org_id,
    'role', om.role,
    'org_name', o.name,
    'org_slug', o.slug,
    'org_status', o.status,
    'plan', o.plan
  ))

```

```

    )) AS memberships
FROM users u
JOIN org_members om ON om.user_id = u.user_id AND om.is_active
JOIN organizations o ON o.org_id = om.org_id
WHERE u.email = lower($1)
GROUP BY u.user_id;

```

### Q3 — Quota Check: Is Org Within API Limit?

```

SELECT
    ucp.quantity AS used,
    p.api_call_limit AS limit,
    ucp.quantity < p.api_call_limit AS within_limit
FROM usage_current_period ucp
JOIN subscriptions s ON s.org_id = ucp.org_id
JOIN plans p ON p.plan_id = s.plan_id
WHERE ucp.org_id = $1
    AND ucp.metric = 'api_calls'
    AND ucp.period_start = DATE_TRUNC('month', CURRENT_DATE)::DATE;

```

### Q4 — Increment Usage Counter (called on every API request)

```

INSERT INTO usage_current_period (org_id, metric, period_start, quantity)
VALUES ($org_id, 'api_calls', DATE_TRUNC('month', CURRENT_DATE)::DATE, 1)
ON CONFLICT (org_id, metric, period_start)
DO UPDATE SET
    quantity = usage_current_period.quantity + 1,
    updated_at = now();

```

### Q5 — Subscription Renewal Query (billing job)

```

SELECT
    s.sub_id, s.org_id, s.plan_id,
    s.billing_interval, s.quantity,
    o.name AS org_name,
    pm.provider, pm.provider_pm_id,
    p.price_cents, p.currency
FROM subscriptions s
JOIN organizations o ON o.org_id = s.org_id
JOIN plans p ON p.plan_id = s.plan_id
LEFT JOIN payment_methods pm ON pm.org_id = s.org_id AND pm.is_default
WHERE s.status = 'active'
    AND s.current_period_end <= CURRENT_DATE + INTERVAL '1 day'
    AND s.cancel_at_period_end = FALSE
ORDER BY s.current_period_end ASC
LIMIT 500;    -- Process in batches

```

## Q6 — Org Usage Report (billing invoice generation)

```
SELECT
    us.metric,
    SUM(us.total_quantity) AS total,
    MIN(us.period_start) AS from,
    MAX(us.period_end) AS to
FROM usage_summaries us
WHERE us.org_id = $1
    AND us.period_start >= $period_start
    AND us.period_end <= $period_end
GROUP BY us.metric;
```

## Q7 — Organization Settings Retrieval

```
SELECT
    o.org_id, o.name, o.slug, o.status, o.plan,
    o.settings, o.timezone,
    s.current_period_end, s.cancel_at_period_end,
    p.seat_limit, p.api_call_limit, p.storage_gb_limit,
    (SELECT COUNT(*) FROM org_members WHERE org_id = o.org_id AND is_active) AS
seat_count
FROM organizations o
JOIN subscriptions s ON s.org_id = o.org_id
JOIN plans p ON p.plan_id = s.plan_id
WHERE o.org_id = $1;
```

## Q8 — Audit Log for Org

```
SELECT
    ae.event_id, ae.action, ae.resource_type,
    ae.resource_id, ae.occurred_at,
    ae.ip_address, ae.metadata,
    u.full_name AS actor_name, u.email AS actor_email
FROM audit_events ae
LEFT JOIN users u ON u.user_id = ae.actor_id
WHERE ae.org_id = $1
    AND ae.occurred_at >= NOW() - INTERVAL '30 days'
ORDER BY ae.occurred_at DESC
LIMIT 100 OFFSET $2;
```

## Q9 — All Flags for an Org (cache warming)

```
SELECT
    ff.key,
    ff.flag_type,
    COALESCE(fo.value, ff.default_value) AS effective_value,
```

```

        ff.rollout_type,
        fo.reason AS override_reason
FROM feature_flags ff
LEFT JOIN flag_overrides fo
    ON fo.flag_id = ff.flag_id
    AND fo.org_id = $org_id
    AND (fo.expires_at IS NULL OR fo.expires_at > now())
WHERE ff.is_active = TRUE
ORDER BY ff.key;

```

## Q10 — Subscription Upgrade Impact (what changes?)

```

WITH current_plan AS (
    SELECT p.* FROM plans p
    JOIN subscriptions s ON s.plan_id = p.plan_id
    WHERE s.org_id = $org_id
),
new_plan AS (
    SELECT * FROM plans WHERE plan_id = $new_plan_id
)
SELECT
    cp.tier AS current_tier,
    np.tier AS new_tier,
    np.price_cents - cp.price_cents AS price_diff_cents,
    np.seat_limit - cp.seat_limit AS seat_limit_change,
    np.api_call_limit - cp.api_call_limit AS api_limit_change,
    np.storage_gb_limit - cp.storage_gb_limit AS storage_change
FROM current_plan cp, new_plan np;

```

## Scaling Strategy

### The Noisy-Neighbor Problem

In shared-schema multi-tenancy, one large tenant's heavy queries can degrade performance for all others. Mitigations:

1. **Resource groups / query timeouts per org:** Use PostgreSQL `statement_timeout` set per connection, limiting query runtime for any single request.
2. **Rate limiting at the API layer:** Before queries reach PostgreSQL.
3. **Partial indexes by org\_id:** Ensure no query can do a full table scan by making `org_id` mandatory in every query.

### Phase 1: Shared Schema (0–10K orgs)

- Single PostgreSQL instance + read replica
- Redis for feature flag caching and usage counters
- All orgs share tables; `org_id` is everywhere

### Phase 2: Tenant Tiering (10K–100K orgs)

- Enterprise orgs (top 1%) get dedicated schemas or databases
- Smaller orgs remain in shared schema

- Introduce connection pooling partitioned by org tier
- usage\_events partitioned monthly; summaries job aggregates hourly

### Phase 3: Horizontal Sharding (100K+ orgs)

- Shard by org\_id hash across multiple PostgreSQL instances
- Use Citus (PostgreSQL extension) for transparent sharding
- Each shard handles ~10K orgs
- Cross-shard queries avoided: all org data collocated

### Usage Metering Architecture

```
API Request → increment Redis counter (INCR org:{id}:api_calls:2024-01)
               → async write to usage_events (Kafka consumer)
               → hourly rollup job updates usage_summaries
               → billing period job reads usage_summaries for invoice
```

This decouples quota enforcement (Redis — sub-millisecond) from billing accuracy (PostgreSQL — eventually consistent).

### Tradeoffs

| Decision                    | Chosen                   | Alternative                    | Why                                                                                                  |
|-----------------------------|--------------------------|--------------------------------|------------------------------------------------------------------------------------------------------|
| Shared schema               | One table, org_id column | Schema per tenant              | Simpler operations, zero schema migration per new tenant; trade-off is isolation and noisy-neighbor  |
| RLS for isolation           | PostgreSQL RLS           | Application-level WHERE clause | Defense in depth; if app forgets WHERE org_id, RLS catches it                                        |
| Usage counters in Redis     | Redis + async Postgres   | Postgres only                  | Postgres can't sustain 50K increments/sec per API endpoint; Redis INCR is atomic and sub-millisecond |
| Feature flags in DB         | PostgreSQL table         | Hard-coded / Config file       | DB enables runtime changes without deployments; cache in Redis for evaluation speed                  |
| Soft deletes on org_members | is_active flag           | Hard DELETE                    | Preserves audit trail of who was ever a member; hard DELETES would orphan audit logs                 |

### Interview Discussion Points

1. **What are the three multi-tenancy models?** (a) Separate database per tenant — best isolation, expensive to operate; (b) Separate schema per tenant — medium isolation, PostgreSQL supports it natively; (c) Shared schema — most efficient, requires org\_id everywhere and RLS for isolation. Most SaaS products start with shared schema.
2. **How do feature flags prevent deployments from being risky?** Feature flags decouple deployment from release. Code ships to production gated behind a flag. The flag is enabled for 1% of orgs

(percentage rollout), observed, then gradually rolled to 100%. Rollback = flip the flag. No re-deployment.

3. **How does usage metering tie into billing?** At billing time, the billing service queries `usage_summaries` for the billing period (filtered by `period_start/end`) to compute overage charges. The subscription base fee comes from the plan. Invoice line items are generated from both.
4. **How do you ensure a user can't access another tenant's data?** Multiple layers: (1) API always filters by the authenticated user's `org_id`; (2) PostgreSQL RLS enforces `org_id = current_setting('app.current_org_id')` at the database level; (3) Integration tests specifically test cross-tenant access paths.

---

## Common Interview Follow-ups

**Q: How do you handle a free-to-paid conversion?** A: Update `organizations.plan`, insert a `subscription_history` record, create a `billing_period`, and charge the payment method. All in a single transaction. The `subscriptions.status` changes from `trial` to `active`.

**Q: How do you enforce seat limits?** A: Before adding a member to `org_members`, query `usage_current_period` where `metric = 'seats_used'`. If `count >= plan's seat_limit`, return a 402 error with upgrade prompt.

**Q: What happens when a subscription lapses?** A: A billing job sets `organizations.status = 'past_due'` after the first failed payment. After 3 retries over 7 days, status becomes `suspended`. All API requests for the org return 402. Data is retained for 30 days before deletion.

---

## Performance Considerations

- **RLS overhead:** PostgreSQL RLS adds a filter to every query. Benchmark shows ~5% overhead with a simple `org_id = $param` policy — acceptable.
- **Usage counter hot path:** Never write `usage_events` synchronously in the API request path. Use Redis INCR and `async flush`. The `usage_current_period` table is for enforcement, not exact billing.
- **Feature flag caching:** Load all flags for an org at session start into Redis. TTL of 60 seconds. Invalidate on flag change. Avoid per-request DB queries for flag evaluation.
- **Audit log write volume:** 100M events/day × 400B = 40GB/day. Monthly partitions; auto-archive after 2 years. Consider TimescaleDB extension for automatic compression.
- **Connection pooling:** With 100K orgs, peak may be 50K concurrent users. PgBouncer in transaction mode with `max_client_conn=50000`, `max_pool_size=200`.

---

## Cross-References

- See `04_banking_platform.md` for payment processing and invoice patterns
- See `21_System_Design/07_analytics_platform_design.md` for usage event analytics
- See `21_System_Design/08_system_design_framework.md` for interview framework
- Row-Level Security: Chapter 8 of this guide
- Partitioning: Chapter 5 of this guide